МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)
Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №13 по курсу
«Информационный поиск»

Текстовое ранжирование TF-IDF

Студент: К. А. Арусланов
Группа: М8О-403Б-23
Преподаватель: А. А. Кухтичев
Дата: 08.11.2026
Оценка:

Москва, 2026

Содержание

1 Постановка задачи **3**

2 Метод решения **3**
2.1 От булева поиска к весам терминов 3
2.2 Схема lnc.ltc и косинусная нормировка 3
2.3 Построение и использование норм 4
2.4 Поиск и отбор top-k 4

3 План и результаты тестирования **5**

4 Результаты **5**
4.1 Корпус и условия измерений 5
4.2 Время поиска 6
4.3 Примеры выдачи и оценка по заголовкам 6
4.4 Воспроизводимость 8

5 Вывод **9**

1 Постановка задачи

Добавить в поисковую систему ранжирование документов на основе TF-IDF. Вместо порядка по идентификатору выдавать сначала документы с большей оценкой соответствия запросу. Реализовать запросы из обычных слов, ограничение выдачи top-k, интерфейс командной строки и веб-форму. Проверить численную правильность оценок, измерить время и дополнительный объём индекса, рассмотреть изменение первых результатов.

Основные структуры и алгоритмы поиска реализуются на C++ без STL-контейнеров. Python используется для веб-обвязки, запуска экспериментов и подготовки таблиц. Входные и выходные данные имеют кодировку UTF-8.

2 Метод решения

2.1 От булева поиска к весам терминов

Булев поиск определяет, подходит документ запросу или нет. Если результатов много, порядок по ID не помогает выбрать наиболее подходящие публикации. Для ранжирования документ и запрос представлены разреженными векторами терминов. Выбрана схема `lnc.ltc` из лекции о текстовом ранжировании (`lesson-09`, слайды 41–43).

Частота $tf_{t,d}$ равна числу вхождений термина t в документ d. Координаты из ЛР10 позволяют посчитать её без повторного чтения корпуса: вхождения в заголовке и основном тексте складываются. В этой работе поля имеют одинаковый вес. Логарифмическая частота задаётся как

$$L(x) = \begin{cases} 1 + \log_{10} x, & x > 0, \\ 0, & x = 0. \end{cases}$$

Так повторение слова увеличивает вес, но увеличение частоты в десять раз не означает десятикратного увеличения вклада в оценку.

Документная частота df_t — число документов, содержащих термин, а N — число всех документов снимка, включая пустые. Обратная документная частота равна $idf_t = \log_{10}(N/df_t)$. Она отличается от общего числа вхождений слова в корпус. Редкий термин имеет больший IDF; термин с $df_t = N$ имеет нулевой IDF.

2.2 Схема lnc.ltc и косинусная нормировка

В обозначении `lnc.ltc` три символа до точки относятся к документу, три после неё — к запросу. `l` означает логарифмический TF, `n` — отсутствие множителя IDF, `t` — IDF, `c` — косинусную нормировку. Таким образом,

$$w_{t,d} = L(tf_{t,d}), \qquad w_{t,q} = L(tf_{t,q})\, idf_t,$$

$$\|d\| = \sqrt{\sum_{t \in V} w_{t,d}^2}, \qquad \|q\| = \sqrt{\sum_{t \in q} w_{t,q}^2}, \qquad score(d,q) = \frac{\sum_{t \in q \cap d} w_{t,d} w_{t,q}}{\|d\|\,\|q\|}.$$

Норма документа считается по *всему* словарю пространства V. Если ограничить её словами запроса, дополнительные слова документа перестанут влиять на нормировку, и получится другая модель. Оценка является косинусным сходством преобразованных векторов, а не вероятностью релевантности.

В тексте слайда 42 нормировка описана неточно; в реализации используется значение `c` из таблицы на слайде 41 и нормированный пример слайда 43. Словам запроса, отсутствующим в словаре корпуса, не соответствует ось: они исключаются из расчёта. Если после этого норма запроса равна нулю, выдача пуста. Документы с нулевой оценкой не включаются в результат.

2.3 Построение и использование норм

Для каждого термина исходного координатного индекса последовательно читаются его вхождения и группируются по документам. По полученному TF к сумме квадратов соответствующего документа добавляется $L(tf)^2$. Одновременно проверяется совпадение DF с булевым индексом. После полного прохода из сумм извлекаются квадратные корни.

Один физический индекс содержит пространства словоформ и лемм. Нормы для них считаются отдельно. При неоднозначной лемматизации каждая различная альтернатива является отдельным термином, как в предыдущих работах. Если альтернативы нескольких слов запроса совпали, они объединяются в одну ось с суммированием query TF.

Расширение хранит два массива норм double и заголовок 80 байт. В нём записаны версия формулы, размеры, FNV-64 и привязка к исходным заголовкам индекса. Расширение совместимо с исходной и соответствующей упакованной копией. При открытии проверяются привязка, размеры, контрольная сумма и допустимость всех значений. Незавершённая сборка помечается `INCOMPLETE` и не открывается.

2.4 Поиск и отбор top-k

Запрос токенизируется; при необходимости выполняется лемматизация. После объединения одинаковых терминов вычисляются веса и норма запроса. Координаты каждого его термина читаются последовательно. Для очередного документа подсчитывается TF и увеличивается накопленная оценка. Затем оценки делятся на заранее рассчитанные нормы документов.

Накопитель содержит N значений double. Для top-k используется собственная ограниченная двоичная куча: в корне находится худший из отобранных результатов. Лучший кандидат заменяет его с восстановлением кучи. В конце куча сортируется по убыванию оценки; при точном равенстве раньше идёт меньший исходный ID. Поддерживаются произвольные 64-битные ID, включая максимальное значение.

Память основных массивов составляет $16N$ байт норм и $8N$ байт оценок, то есть 11,49 МБ на данном корпусе. Дополнительно нужны буферы, словарь лемматизатора и $O(k)$ памяти результатов. Время состоит из чтения координат терминов запроса, обхода массива $O(N)$ и отбора $O(M \log k)$, где M — число документов с положительной оценкой. Полный словарь и координатный индекс в память не загружаются.

В CLI выбран режим `-ranking tfidf`; запросы читаются по строкам stdin, ответы выдаются в JSONL. Поле `score` содержит оценку, `total` — полное число положительных совпадений, `-limit` ограничивает только число показанных результатов. Веб-форма вызывает тот же C++-поиск и позволяет выбрать способ выдачи. В режиме TF-IDF слова образуют «мешок слов»: порядок и кавычки не задают обязательную фразу. Булев и цитатный поиск доступны отдельно.

3 План и результаты тестирования

Для численной проверки используется небольшой корпус с заранее известными частотами. Независимый эталон в GoogleTest строит полные векторы из текстов, вычисляет DF, TF-IDF и косинус, а затем полностью сортирует результаты. Основной алгоритм сравнивается с ним с допуском 10^{-12}.

План включает следующие группы проверок:

1. Однословные и многословные запросы, повторение слов, неизвестные слова, одинаковые частоты и объединение вхождений заголовка и текста.

2. Норма по всему словарю, включая слова вне запроса; влияние лишних слов на итоговую оценку документа.

3. Точное совпадение top-k с префиксом полной сортировки, $k = 0$, равные оценки и большие ID. Отдельный корпус из 1200 документов проверяет кучу для $k = 1, 7, 20, 1000$.

4. Пустой корпус, пустые документы, отсутствие совпадений и запрос только из терминов с нулевым IDF: отсутствие деления на ноль и NaN.

5. Отдельные нормы лемм, неоднозначные разборы и объединение альтернатив в запросе; идентичность оценок исходного и сжатого индексов.

6. Повреждённые, усечённые, чужие и незавершённые нормы; недопустимый UTF-8, ограничения длины, последовательность запросов с восстановлением после ошибки.

7. Передача параметров из веб-формы, UTF-8, вывод оценок и выбор TF-IDF через настоящий HTTP-запрос к тестовому серверу.

Группа ЛР13 содержит 11 GoogleTest, проверку CLI help и 3 Python-теста. Полная регрессия завершилась успешно: 144 CTest и 286 Python-тестов; 6 интеграционных проверок MongoDB пропущены. Журнал прогона приложен в `lab13/tests.txt`.

4 Результаты

4.1 Корпус и условия измерений

Использован тот же снимок из 478 833 игровых публикаций, что в предыдущих работах. В двух пространствах суммарно 2 545 616 терминов и 335 548 564 координаты. Построение норм заняло 51,140 с, включая полную проверку исходных булева и координатного индексов. Это один запуск, а не медиана. Размер расширения — 7 661 408 байт, около 7,66 МБ.

Поиск выполнялся по упакованному VB-индексу. Для измерения времени в каждом пространстве сравнивались OR с порядком по ID и TF-IDF по тем же словам запроса. OR выбран здесь потому, что для заданных запросов он перебирает тот же набор документов, которым TF-IDF назначает положительную оценку. Для сравнения первых результатов дополнительно выполнен более строгий булев запрос с AND; его число найденных документов показано отдельно. Проведено семь раундов, порядок режимов и запросов чередовался. Каждый процесс выполнял прогревочный

пакет и затем измеряемый пакет. Кэш ОС не очищался. Приведены медианы времени внутри C++: нормализация запроса, чтение списков, расчёт оценок и отбор результатов. Открытие индекса, загрузка морфологии и норм, чтение метаданных выдачи и вывод JSON в это время не входят.

4.2 Время поиска

В таблице приведены результаты для словоформ. Для выбранных запросов число кандидатов OR совпало с числом положительных оценок TF-IDF; оно не зависит от ограничения top-10.

Запрос	Кандидатов	OR/ID, мс	TF-IDF, мс
doom eternal	9 170	0.892	5.982
god of war	228 464	15.757	135.160
cyberpunk 2077	8 407	1.062	5.601
baldur gate	7 173	0.936	4.013
игры	149 495	6.066	26.313
игрой	22 588	1.150	4.387
несуществующийтерминхуz	0	0.177	0.170

Ранжирование требует больше работы, чем перечисление булева объединения: нужно восстановить TF из координат и вычислить оценки. Особенно заметна разница на `god of war`, где частое слово `of` порождает длинный список. Поэтому TF-IDF здесь улучшает порядок выдачи ценой времени поиска.

Для лемм запросы «игры» и «игрой» дали одинаковое число кандидатов 230 075; медианы составили 67,103 и 63,765 мс соответственно. По словоформам это разные запросы: 149 495 и 22 588 кандидатов. Все оценки и top-10 сохранялись между повторениями; исходный и сжатый форматы дополнительно сопоставлены в численных автотестах.

4.3 Примеры выдачи и оценка по заголовкам

Для запроса `doom eternal` в TF-IDF первыми стали:

Заголовок	Оценка
Doom Eternal — Мнение	0,81278
Сравнение графики Doom vs Doom Eternal	0,67033
5 подтвержденных оружий в Doom Eternal	0,57472
Doom Eternal — 20 минут захватывающего геймплея	0,54836
Fallout 76 — Новая система перков и отличный DOOM Eternal	0,44219

Более строгий запрос `doom AND eternal` нашёл 1 392 документа против 9 170 документов с положительной оценкой TF-IDF. Среди первых пяти результатов AND по ID два заголовка прямо называли Doom Eternal, в первых пяти TF-IDF — все пять. Таким образом, оба режима способны найти публикации по игре, но формируют разные первые результаты. Общее число результатов здесь нельзя считать показателем релевантности: AND требует оба слова, а TF-IDF допускает одно совпадение.

Для небольшой иллюстративной оценки взяты три запроса по названиям игр. Помощником вручную размечено объединение первых пяти результатов AND/ID и TF-IDF по видимым заголовкам. Метка 1 означает, что заголовок прямо называет

искомую игру, включая русское написание; метка 0 — что тема из заголовка не установлена. Для каждого режима рассчитана доля меток 1 в первых пяти результатах (P@5).

Запрос	AND/ID	TF-IDF
doom eternal	0,40	1,00
god of war	0,80	1,00
cyberpunk 2077	0,60	1,00
Среднее	0,60	1,00

Это оценка соответствия заголовков теме на трёх примерах. Полные тексты не размечались, запросы не составляют представительную независимую выборку. По этим результатам нельзя заключить, что общая точность поиска равна 1. Разметка сохранена вместе с ID и заголовками, поэтому расчёт можно повторить.

На рис. 1 и 2 показан ещё один реальный запрос в пространстве лемм. Булев режим выполняет `чит AND коды AND gta` и находит 6 документов. Режим TF-IDF обрабатывает `чит коды gta` как запрос из слов и находит 18 743 документа с положительной оценкой. В булевой первой пятёрке по ID есть «Gears of War», «Наши истории» и статья о чит-кодах вообще; у TF-IDF первыми идут публикации, в заголовках которых упоминаются GTA и читы или коды. Один документ, ID 431 639, присутствует в обеих выдачах; TF-IDF ставит его на первое место. Скриншоты показывают порядок и число результатов в отдельных запусках веб-интерфейса; время на них не является медианой измерений из предыдущей таблицы.

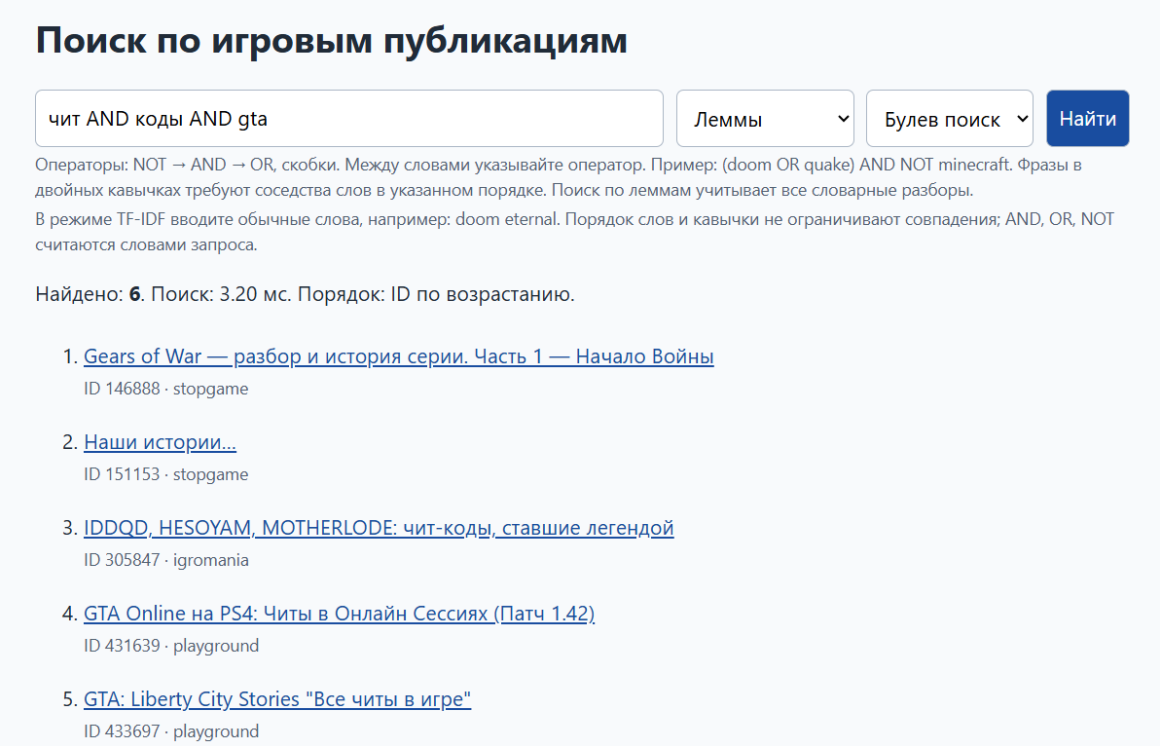


Рис. 1 — Булев запрос по леммам: все три слова обязательны, выдача по ID

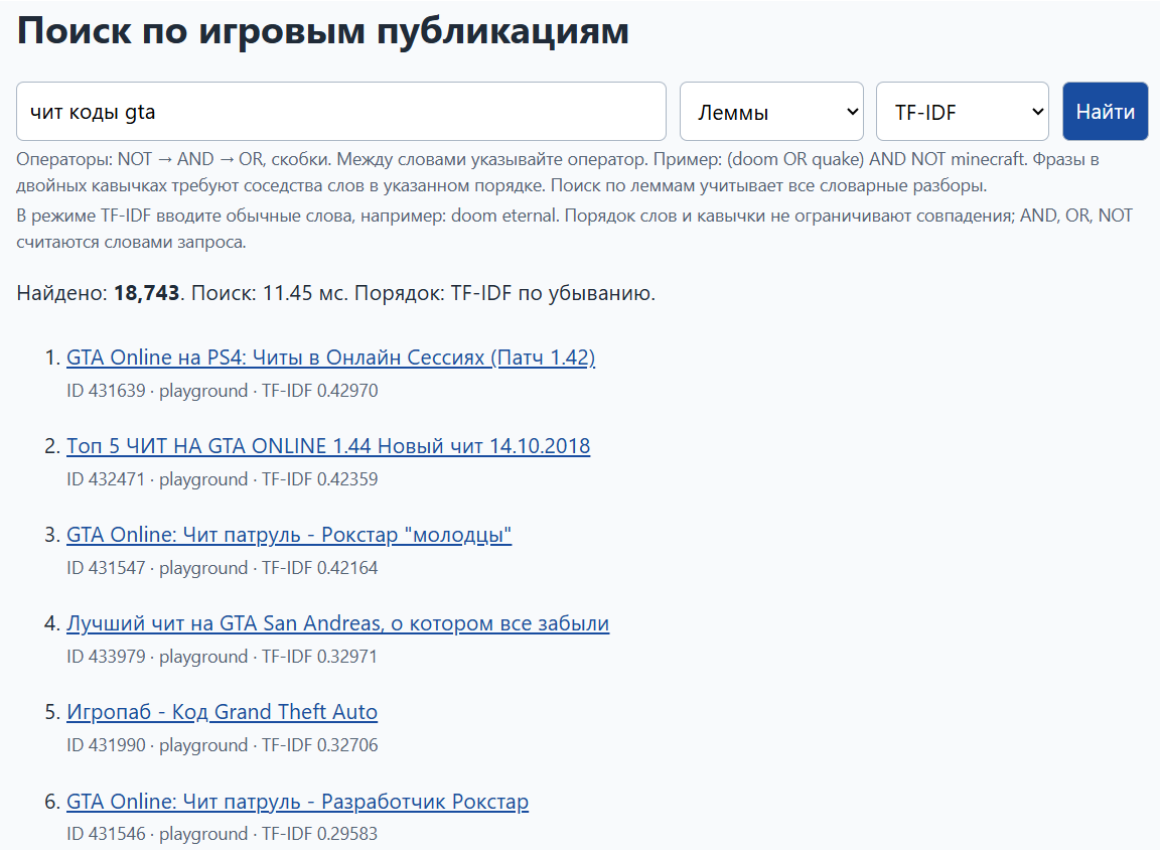


Рис. 2 — Запрос TF-IDF по леммам: 18 743 документа с положительной оценкой

Первые пятёрки TF-IDF в этих примерах состоят из публикаций `Playground`. Это также требует внимания: косинусная модель может высоко оценивать короткие тексты с большой долей слов запроса. Высокая оценка не означает полноту, актуальность или достоверность материала. Разнообразие источников, качество содержимого и дату эта модель не учитывает.

4.4 Воспроизводимость

Команды сборки, построения норм, CLI и веб-формы приведены в `lab13_run.md`. Скрипт `../run_lab13.py` сохраняет 196 измерений, top-10, медианы, SHA-256 входов и результаты оценки по сохранённой разметке. Исходные данные эксперимента находятся в `lab13/`.

Исходный код: github.com/kirbasss/MAI-IR/tree/lab13.

Рис. 3 — Ссылка на ветку лабораторной работы №13

5 Вывод

В этой работе я добавил ранжирование TF-IDF по схеме `lnc.ltc`. Поисковая система принимает обычные слова и возвращает top-k документов с числовыми оценками. Независимые автотесты подтвердили расчёт косинуса, порядок выдачи и одинаковую работу с исходным и сжатым индексами.

На рассмотренных запросах TF-IDF поднял тематические публикации в первые строки выдачи даже при сравнении с более строгим булевым AND. При этом поиск стал дороже: например, для `god of war` медиана увеличилась с 15,757 до 135,160 мс. Нормы потребовали всего 7,66 МБ, но извлечение частот из длинных координатных списков занимает время. Его можно уменьшить, если сохранять TF непосредственно в постингах.

Полученная модель пока не учитывает порядок слов, разные веса полей и контекст неоднозначных лемм. В выдаче заметно преимущество коротких публикаций; небольшая оценка по заголовкам не заменяет разметку полных документов. Дальше имеет смысл расширить набор запросов, сравнить модели нормировки и отдельно исследовать влияние длины текста на результат.